

Take-home test: Kitchen Display Queue

1) What this is

Build a small internal tool for a restaurant kitchen. The **expeditor** (the person who decides what the kitchen works on next) needs a screen that shows every active order ranked by priority, and lets them move each order through its preparation stages.

This exercise is intentionally small. We expect you to use an AI assistant (Cursor, Claude, or similar). We are not grading how much code you can produce. We are grading whether the result is well-organized, whether the business rules live in the right layer, and whether you can explain the decisions you made.

In the technical panel you will walk us through your decisions, and you will change the running code in response to a product request we have not shared here. Design the project so that a change to ranking or to status rules is local and quick.

Time box: 4 hours, stop when you hit it. A smaller result with an honest decision log is a better submission than a wide one you cannot explain.

2) AI

You may use any AI tool for any part of this exercise.

You must be able to explain every file you submit, including code you did not type yourself. During the panel we will open specific functions and ask why they are written that way. A file you cannot explain works against you.

In `DECISIONS.md`, name one AI suggestion you kept and one you rejected or rewrote. Be specific: quote the idea, not the tool brand

Include your AI chat logs in the repository. Create a folder called `ai-logs/` at the root and export the full conversation history from the tool you used (screenshots, markdown exports, or `.json` files — whatever your tool supports). We will not penalize you for wrong turns or failed attempts in the logs; we want to see how you steered the tool, what you asked, and when you course-corrected.

3) Domain

A restaurant receives orders throughout the day. Each order has a type (dine-in, takeout, or delivery), a list of menu items, and optionally a promised time. The kitchen does not prepare orders in arrival order; instead, a priority score determines the queue. The expeditor uses this screen to see what to work on next and to advance each order through its stages.

No authentication, no payments, no real customer data.

4) Data Model

4.1) Menu item (provided by seed, no CRUD needed)

FIELD	RULES
name	Unique, non-empty
category	starter , main_course , dessert , or drink
prep_time_minutes	Integer > 0

4.2) Order

FIELD	RULES
customer_name	Non-empty string
type	dine_in , takeout , or delivery
placed_at	Timestamp, set on creation
promised_at	Timestamp or null. Relevant for delivery and takeout .
is_vip	Boolean
status	received , preparing , ready , picked_up , or cancelled

4.3) Order item (join table)

FIELD	RULES
order_id	References an existing Order
menu_item_id	References an existing Menu Item
quantity	Integer > 0

5) Priority Queue

The default queue lists orders in `received` or `preparing` status. Terminal orders (`picked_up` , `cancelled`) and `ready` orders stay out of the active queue.

Sort by priority score, highest first. The score is computed at request time, using the real clock in the running app and a frozen `now` in tests. All time comparisons use UTC.

5.1) Score

INPUT	POINTS
Order type	
dine_in	30

takeout	20
delivery	10
VIP	+20 if <code>is_vip</code> is true
Wait time	<code>min(40, floor(minutes_waiting / 10) * 5)</code>
Promised time	
<code>promised_at</code> is within 30 minutes from now	25
<code>promised_at</code> is within 31–60 minutes from now	15
<code>promised_at</code> is more than 60 minutes away or null	0
Complexity	<code>min(20, floor(total_prep_minutes / 15) * 5)</code>

Where:

- `minutes_waiting` = whole minutes elapsed since `placed_at`, floored. An order placed 9 minutes and 50 seconds ago has 9 whole minutes = 0 wait-time points;
- `total_prep_minutes` = sum of `(prep_time_minutes * quantity)` across all items in the order;

5.2) Tie-break

When two orders have the same score, sort by:

1. Earlier `promised_at` first (`null` goes last);
2. Earlier `placed_at` first;
3. Smaller `id` first;

5.3) Worked Examples

All examples use `now = 2026-06-15T12:00:00Z`.

Example A

A `dine_in` order, not VIP, placed at `2026-06-15T11:25:00Z` (35 minutes ago), no `promised_at`. The order has 2x Grilled Salmon (`prep_time` 20 min) and 1x Caesar Salad (`prep_time` 10 min). Total prep = $(20 * 2) + (10 * 1) = 50$ minutes.

COMPONENT	CALCULATION	POINTS
Type	<code>dine_in</code>	30
VIP	not VIP	0
Wait time	<code>min(40, floor(35 / 10) * 5) = min(40, 15)</code>	15
Promised	null	0
Complexity	<code>min(20, floor(50 / 15) * 5) = min(20, 15)</code>	15

Total		60
-------	--	----

Example B

A `delivery` order, VIP, placed at `2026-06-15T11:50:00Z` (10 minutes ago), `promised_at` = `2026-06-15T12:20:00Z` (20 minutes from now). The order has 1x Margherita Pizza (`prep_time` 12 min). Total prep = 12 minutes.

COMPONENT	CALCULATION	POINTS
Type	<code>delivery</code>	10
VIP	VIP	20
Wait time	<code>min(40, floor(10 / 10) * 5) = min(40, 5)</code>	5
Promised	20 min from now (within 30)	25
Complexity	<code>min(20, floor(12 / 15) * 5) = min(20, 0)</code>	0
Total		60

Both score 60. Tie-break: Example B has a `promised_at` and A does not, so **B ranks above A**.

6) Status changes (state machine)

FROM	TO	RULE
<code>received</code>	<code>preparing</code>	Allowed
<code>received</code>	<code>cancelled</code>	Allowed
<code>preparing</code>	<code>ready</code>	Allowed
<code>ready</code>	<code>picked_up</code>	Allowed
anything else	anything else	Rejected

`picked_up` and `cancelled` are terminal. No order can skip a step (`received` cannot jump to `ready`). Repeating the same transition (e.g. acknowledging an already `preparing` order) is an error, not a silent success.

Enforce this on the server, hiding a button in the UI is not enough. If someone calls the API directly, the server must reject an illegal transition with a clear error.

7) API

METHOD	PATH	BEHAVIOR
<code>GET</code>	<code>/orders/queue</code>	Default: <code>received</code> and <code>preparing</code> , sorted by score. Optional query param <code>status=received</code> or <code>status=preparing</code> to filter. Each order in the response includes its computed <code>score</code> .

POST	/orders/:id/start	Transition: received → preparing .
POST	/orders/:id/ready	Transition: preparing → ready .
POST	/orders/:id/pickup	Transition: ready → picked_up .
POST	/orders/:id/cancel	Transition: received → cancelled .

For illegal transitions or other validation errors, return a `4xx` response with a stable, machine-readable error code (e.g. `INVALID_TRANSITION` , `ORDER_NOT_FOUND`). Choose your codes and document them briefly in the README.

8) UI

One page: the Kitchen Display.

- A table showing the active queue: score, customer name, order type, items summary, wait time, promised time, VIP badge, status;
- A way to filter by `received` , `preparing` , or the default active queue;
- Actions to advance each order to its next status. Only show the action that is valid for the current status;
- A way to cancel a `received` order;
- An empty state when the queue has no orders;
- When the API rejects a transition, the error must be visible on the screen (not just in the browser console);

Use **MUI**. Match the layout to a dense internal tool (think ops dashboard), not a marketing page.

The **score and the sort order must come from the API**. The UI must not contain a second copy of the scoring formula. The table displays whatever the API returns, in the order the API returns it.

9) Technical constraints

One repository, two apps (API and web). Do not depend on any private or external repository.

9.1) API

- Node.js 22;
- JavaScript;
- Express;
- Sequelize 6, with migrations (do not rely on `sync()` for schema creation);
- PostgreSQL 16;
- Jest;

Organize the API in layers:

LAYER	RESPONSIBILITY
Endpoint	HTTP only. Read the request, call the module, write the response. Thin.
Module	Business logic: priority scoring, state machine transitions.
Repository	Data access: Sequelize queries, includes, filters.
Schema	Sequelize models and migrations.

The priority function must be pure. It receives the order data and `now` as arguments, and returns a number. Its Jest tests run without PostgreSQL and without mocking Sequelize.

9.2) Web

- Vite;
- React 19;
- TypeScript;
- MUI 7;
- Vitest (for extra points only; see below);

9.3) Database

Ship a `docker-compose.yml` that starts PostgreSQL 16 only. Use host port 5433 so a local Postgres on 5432 does not collide.

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: kitchen_queue
      POSTGRES_PASSWORD: kitchen_queue
      POSTGRES_DB: kitchen_queue
    ports:
      - "5433:5432"
```

9.4) Getting started

From a clean machine, the README should guide someone through:

1. `docker compose up -d` (start Postgres)
2. One install command
3. One `db:setup` command (runs migrations + seed)
4. One `dev` command (starts both the API and the web app)

Suggested ports: API on `4000`, web on `5173`.

9.5) Seed

You write the seed, it must contain **menu items** and **orders with order items**.

Seed dates must be **relative to the UTC time when the seed runs** (e.g. "placed 2 hours ago", "promised in 20 minutes"). Hard-coded calendar dates go stale and break the demo.

Include enough rows to demonstrate:

- Every order type (`dine_in` , `takeout` , `delivery`)
- At least one VIP and one non-VIP order
- A wait-time cap (an order old enough that wait-time points hit the maximum)
- A complexity cap (an order with enough total prep time to hit the maximum)
- A tie on score broken by `promised_at`
- Orders in different statuses (`received` , `preparing` , and at least one `ready` that is not in the active queue)

Tests do not use the seed, they pass their own fixtures and a frozen `now` .

10) Tests (Required)

FRAMEWORK	WHAT IS PROVES
Jest (priority)	Score calculation, both caps (wait-time and complexity), all <code>promised_at</code> buckets (within 30 min, within 60 min, beyond, null), tie-break, and at least one case where a lower base-weight type wins on the rest of the formula (e.g. a <code>delivery</code> VIP with a tight promise beats a <code>dine_in</code> non-VIP).
Jest (transitions)	Every legal transition succeeds. Illegal transitions are rejected (skipping a step, repeating the same status, transitioning from a terminal status). A cancelled or picked-up order does not appear in the default queue.

Priority tests must freeze `now` . A test that calls `Date.now()` and hopes the clock still matches is a failing submission.

11) Extra points

Do these only after the required core is solid and you can fully explain it. Skipping all of them does not count against you. In `DECISIONS.md` , note which extras you attempted and which you left out.

11.1) POST /orders endpoint

API only, no UI form needed. Validate that `items` is non-empty, each `menu_item_id` references an existing menu item, and each `quantity` is a positive integer. Include at least one Jest test for a validation error.

11.2) Vitest component test

The cancel button is only shown for `received` orders; or the queue table renders an empty state when there are no orders.

11.3) Playwright end-to-end

One happy path: the queue renders a seeded order, the expeditor starts preparation, and the status changes on screen. Wire it with Playwright's `webServer` config so `npm run test:e2e` boots what it needs.

12) What to submit

A git repository we can clone, with:

- `README.md` — prerequisites (Docker, Node 22), the four commands to get running, ports, and your API error codes;
- `DECISIONS.md` — the sections listed below;
- The API, the web app, migrations, seed, and tests;

12.1) DECISIONS.md sections

Write a few short paragraphs for each. We read these before the panel.

1. Scope

- What you built and what you cut because of the 4-hour box. Which extra points you attempted and which you skipped;

2. Layers

- What lives in the module, the repository, and the UI. Name one rule you refused to put in the UI and why;

3. Priority

- How the score is produced at request time, and one edge case that was easy to get wrong;

4. Transitions

- Where the state machine is enforced, and what happens when someone tries to start an order that is already `preparing`;

5. AI

- One AI suggestion you kept and one you rejected or rewrote. Quote the idea and explain your reasoning;

6. Next

- If this had to run in a real kitchen next month, name the first two changes you would make. Do not implement them;

13) The Interview

Plan on a short walkthrough of your `DECISIONS.md` and the priority rule, followed by a live change to this repository. The change will be a product tweak to ranking or status, in the area you already built. We will **not ask you to re-architect the app**, and we will **not ask you to implement the extra-point items**.

Arrive with Docker running and both apps able to boot in a couple of minutes. We would rather watch you change the code than watch you install dependencies.

14) Out of Scope

Authentication, user accounts, and roles. Payments or billing. A real customer entity (just a name string on the order). Pagination. A design-system package, Storybook, or CI pipeline. Restify, Redis, or background jobs. Real-time updates (WebSockets). Pixel-perfect visuals.

You may deviate from the suggested layout below. If you do, explain why in `DECISIONS.md`.

```
api/  
  src/  
web/  
  src/  
ai-logs/  
docker-compose.yml  
README.md  
DECISIONS.md
```